

claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf\_f6b96576-946\agent-a8934b1ab827cb493.jsonl`
- SHA-256: `d82d0755885fc2cef1f3c7be096118f665a40ea33460cd1a95108b235b521861`
- Source modified: `2026-07-06T18:02:58+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf\_f6b96576-946`
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`

Transcript

1. user (2026-07-06T18:00:43.631Z)

You are a CORRECTNESS reviewer focused on the F3 all-errored-handoff logic.

REPO ROOT: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer

FULL DIFF UNDER REVIEW (read first): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-

indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_full.diff

BACKGROUND: wf294 = the Cloud Run rally-worker recorded status="success" / 0 segments even when the WASB detector TIMED OUT and produced no trajectory ? a detector failure indistinguishable from

a legit 0-rally video. An earlier change (already reviewed CLEAN) fixed the two core detector paths

(healthcheck-not-ready, no-trajectory-abort) to return backend.results.Failure, and made worker.cmd_infer's _fail() write+push a status="failed" report.json + done-marker.

THIS REVIEW covers the FOLLOW-UP completeness work that closed the same silent-failure CLASS on adjacent paths (a prior review confirmed these were the last gaps). Three sub-changes:

F1 ? backend/pipeline/segmenters/trajectory_hybrid.py process_video: the pre-detection config/env/

input abort paths now return Failure instead of bare []: unknown sport (KeyError), missing API

key (skip_ai=False), unknown provider (KeyError), invalid/<=0 video duration, and _resolve_runner

NotImplementedError (detector_impl=native on a family with no native runner). A genuine empty match (trajectory produced but zero candidate windows) still returns bare [].

F2 ? backend/pipeline/segmenters/yolo_hybrid.py (the model-default segmenter): same treatment ? _resolve_provider now returns Failure (not None) on bad sport / missing key / unknown provider;

process_video returns Failure on invalid duration; return annotation widened; imports Failure.

F3 ? the AI-handoff "all windows errored" path in BOTH trajectory_hybrid.py (inline Phase 3) and yolo_hybrid.py (_run_ai_handoff): the per-window handoff fn now returns (valid_segments, errored)

where errored=True iff the window yielded NO verdict (clip extraction failed OR provider metrics['error']). If candidate windows > 0 AND every handed-off window errored,

process_video

returns a Failure(is_recoverable=True). If ANY window ran (even returning no rallies), it stays a

Success (bare list). skip_ai (fully-local) has no provider and never reaches this path.

CONTRACT: every caller (worker.cmd_infer:326, main.py:1557, orchestration.py:806) branches on isinstance(result, Failure). worker._fail writes status="failed" report.json + marker.

trajectory_hybrid is NOT mypy-quarantined (must stay mypy-clean); yolo_hybrid + worker.__main__ ARE
quarantined (mypy.ini ignore_errors). CI: ruff 0.15.16, mypy 2.1.0 (mypy backend training),
run_all_tests.py, coverage floor 70%. Current state: ruff clean, mypy clean, 1740 passed / 3
skipped, coverage 85.85%. So do NOT speculate that "tests might fail" ? cite concrete code
defects.

Report ONLY real defects with exact file:line: a wrong/edge behavior (off-by-one, an all-errored
run that stays "success", or a partial/legit-empty run wrongly failed), a broken contract, a
thread-safety/ordering bug, a regression vs the prior behavior, a convention violation, or a
test
that asserts the wrong thing / has a gap. If you cannot ground a concern in the actual code, DO
NOT
report it.

YOUR LENS: In BOTH trajectory_hybrid.py (inline Phase 3) and yolo_hybrid.py (_run_ai_handoff),
scrutinize the "all windows errored -> Failure" decision:

- The guard is roughly ``if <windows> and handoff_failed == len(<windows>): return Failure``. In
trajectory it uses `handoff_indices` (the `_select_for_handoff` subset); in yolo it uses
`all_candidate_windows`. Is each the RIGHT denominator? Could an EMPTY handoff set make `0==0`
fire a
`false` Failure? Could a fusion Gate that filters all windows (`handoff_indices` empty while
`all_candidate_windows>0`) mislabel a legit run?
 - errored semantics: `errored=True` on (`extraction-fail`) OR (`metrics['error']`). Is a window that
ran
the provider successfully and returned `[]` correctly counted as NOT errored (a real 'no
rallies'
verdict)? Trace `process_candidate_window`'s returns.
 - Ordering/thread-safety: results are now collected via `window_results = [f.result() for f in
futures]` then iterated. Is submission-order (temporal order) preserved for `ai_segments/
gemini_segments` exactly as before? Any shared-state race in the threads?
 - Does F3 change ANY behavior on the `skip_ai` (fully-local) path or when `handoff_failed <
len(windows)`? Confirm partial-success and all-success are unchanged.
- Report concrete correctness/edge defects with file:line + a failure scenario.

2. assistant (2026-07-06T18:00:46.548Z)

I'll start by reading the full diff under review, then examine the actual F3 code in both files.

3. user (2026-07-06T18:00:48.565Z)

```
1      diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2      index 503f6d8..7987278 100644
3      --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4      +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5      @@ -22,7 +22,7 @@ import concurrent.futures
6      import logging
7      import os
8      import time
9      -from typing import Any, Dict, List, Optional, Tuple
10     +from typing import Any, Dict, List, Optional, Tuple, Union
11
12     import cv2
13
14     @@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
15     from backend.pipeline.detectors.base import DetectorRunner
16     from backend.pipeline.segmenters.base import BasePlaySegmenter
17     from backend.providers import provider_registry
18     +
19     +# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
20     +from backend.results import Failure
21     from backend.sports import sport_registry
22     from backend.utils.paths import output_base
23     from backend.utils.video import extract_video_chunk, get_video_duration
```

```

24     @@ -174,10 +177,14 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
25         video_path: str,
26         player_pool: Optional[List[str]] = None,
27         max_frames: Optional[int] = None,
28     - ) -> List[Dict[str, Any]]:
29     -     # override-ignore: the ABC declares the Result contract (Success|Failure)
30     -     # but every live segmenter still returns a bare list ? the documented
31     -     # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
32     + ) -> "Union[List[Dict[str, Any]], Failure]":
33     +     # Result contract (the ABC declares Success|Failure). Anything that means the
run could
34     +     # NOT actually complete ? an unrunnable config (unknown sport/provider, missing
API key),
35     +     # an unreadable video, an unavailable detector (native-not-supported), the env
healthcheck
36     +     # not ready, or run_predict timing out / aborting with no trajectory ? returns
a ``Failure``
37     +     # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match
(a trajectory
38     +     # was produced but no candidate/AI-confirmed rallies) still returns a bare
``[]``. Every
39     +     # caller (worker.cmd_infer, main.py, orchestration) branches on
isinstance(result, Failure).
40         label = self.display_label
41         # --- Sport profile + AI provider wiring (identical across segmenters) ---
42         sport_name = self.config.sport
43     @@ -185,7 +192,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
44         sport_profile = sport_registry.get(sport_name)
45     except KeyError as e:
46         logger.error(str(e))
47     -     return []
48     +     # Unrunnable config, not a 0-rally match (mirrors the gemini segmenter).
49     +     return Failure(str(e))
50
51         idx_cfg = self.config.indexing
52         skip_ai = bool(idx_cfg.skip_ai_handoff)
53     @@ -205,24 +213,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
54         if not api_key:
55             from backend.pipeline.segmenters._keyhint import api_key_hint
56
57     -         logger.error(
58     +         msg = (
59             f"{api_key_env_var} environment variable is not set! "
60             f"To run the {provider_name} handoff, set your API key. "
61             + api_key_hint(api_key_env_var)
62         )
63     -         return []
64     +         logger.error(msg)
65     +         # A misconfig (unrunnable), NOT a 0-rally match ? surface as Failure so
the run is
66     +         # marked failed instead of a silent empty "success" (mirrors gemini;
the worker's
67     +         # own 0-segment diagnostic names this missing-key case as a top cause).
68     +         return Failure(msg, is_recoverable=True)
69         try:
70             provider = provider_registry.get(
71                 provider_name, api_key=api_key, model_name=model_name
72             )
73         except KeyError as e:
74             logger.error(str(e))
75     -         return []
76     +         return Failure(str(e))
77
78         video_duration = get_video_duration(video_path)
79         if video_duration <= 0:

```

```

80         logger.error("Invalid video duration.")
81     -     return []
82     +     # An unreadable/broken video, not a 0-rally match.
83     +     return Failure(
84     +         message=f"{label}: invalid/unreadable video duration
({video_duration}).",
85     +     )
86     fps, frame_width = self._probe_fps_width(video_path)
87
88     # --- Runner config + windowing thresholds ---
89     @@ -233,9 +248,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
90         runner, runner_params = self._resolve_runner(idx_cfg)
91     except NotImplementedError as e:
92         # e.g. detector_impl="native" for a family without a native runner
93     (tracknet, or
94     -     # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
95     +     # fusion with a tracknet substrate). An unavailable detector is a failure,
not a
96     +     # 0-rally match ? refuse cleanly as a Failure instead of crashing or a
false success.
97     +     logger.error("%s: %s", label, e)
98     -     return []
99     +     return Failure(message=f"{label}: {e}")
100
101     velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
102     merge_gap = idx_cfg.dead_time_merge_gap
103     @@ -267,7 +283,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
104     ok, msg = runner.healthcheck()
105     if not ok:
106         logger.error("%s detector environment not ready: %s", label, msg)
107     -     return []
108     +     # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
109     +     # failed/retryable instead of writing an empty-but-"successful" result.
110     +     return Failure(
111     +         message=f"{label} detector environment not ready: {msg}",
112     +         is_recoverable=True,
113     +     )
114     logger.info("%s detector env OK: %s", label, msg)
115
116     # --- Phase 2: trajectory -> candidate rally windows ---
117     @@ -278,7 +299,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
118     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
119     if not csv_path:
120         logger.error("%s produced no trajectory; aborting.", label)
121     -     return []
122     +     # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
123     +     # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
124     +     # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
125     +     # rather than a silent 0-segment "success" (the wf294 incident).
126     +     return Failure(
127     +         message=(
128     +             f"{label} detector produced no trajectory (timed out or aborted) ?
"
129     +             "detector failure, not a 0-rally video."
130     +         ),
131     +         is_recoverable=True,
132     +     )
133     points = runner.parse_trajectory_csv(csv_path)
134     logger.info(

```

```

135     @@ -387,7 +418,11 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
136         len(handoff_indices),
137     )
138
139     -
140     def process_candidate_window(wi: int, window: Dict[str, Any]) ->
List[dict]:
141     +
142     def process_candidate_window(
143     +     wi: int, window: Dict[str, Any]
144     + ) -> Tuple[List[dict], bool]:
145     +     """Returns (valid_segments, errored) ? errored=True when this window
yielded NO
146     +     verdict (clip extraction failed, or the provider returned
metrics['error'])."""
147     +     w_start = max(0, window["start"] - handoff_padding)
148     +     w_end = min(video_duration, window["end"] + handoff_padding)
149     +     w_dur = w_end - w_start
150     @@ -395,7 +430,7 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
151     if not extract_video_chunk(
152     +     video_path, w_start, w_dur, w_path, reencode=True
153     + ):
154     -     return []
155     +     return [], True # extraction failed ? no verdict for this window
156     +     prompt = sport_profile.get_prompt(chunk_duration=w_dur)
157     +     segments, metrics = provider.analyze_video(
158     +         w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
159     @@ -403,7 +438,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
160     # B2: a provider failure returns [] with metrics["error"] set ? that is
161     # "we don't know", NOT "no rallies in this window". Surface it loudly
162     so a
163     # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
164     -
165     if metrics.get("error"):
166     +     errored = bool(metrics.get("error"))
167     +     if errored:
168     +         logger.warning(
169     +             "[Handoff %d] provider error ? window %d skipped: %s",
170     +             wi + 1,
171     @@ -427,7 +463,7 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
172     os.remove(w_path)
173     except OSError:
174     +     pass
175     -     return valid
176     +     return valid, errored
177
178     with concurrent.futures.ThreadPoolExecutor(
179     +     max_workers=ai_max_workers
180     @@ -438,8 +474,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
181     )
182     +     for wi in handoff_indices
183     + ]
184     -     for future in futures:
185     -         ai_segments.extend(future.result())
186     +     window_results = [future.result() for future in futures]
187     +     for valid, _errored in window_results:
188     +         ai_segments.extend(valid)
189     +     handoff_failed = sum(1 for _valid, errored in window_results if errored)
190     +     try:
191     +         os.rmdir(handoff_dir)
192     +     except OSError:
193     @@ -448,6 +486,18 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
194     "Phase 3 completed. AI confirmed %d semantic play segments.",
195     +     len(ai_segments),
196     + )
197     + # wf294 (all-errored handoff): if EVERY handed-off window failed to yield a
verdict

```

```

194      +          # (provider error / clip extraction failed), the run has zero information ?
a FAILURE,
195      +          # not a 0-rally match. A single window that RAN and returned no rallies is
a legit empty
196      +          # and keeps this from firing. (skip_ai has no provider, so it never reaches
here.)
197      +          if handoff_indices and handoff_failed == len(handoff_indices):
198      +              msg = (
199      +                  f"{label} AI handoff produced no verdict: all {handoff_failed}
candidate "
200      +                  "window(s) errored (provider error / clip extraction failed) ?
incomplete "
201      +                  "run, not a 0-rally match."
202      +              )
203      +              logger.error(msg)
204      +              return Failure(message=msg, is_recoverable=True)
205
206          # --- Phase 4: DB population + candidate linking ---
207          segments_data = []
208      diff --git a/backend/pipeline/segmenters/yolo_hybrid.py
b/backend/pipeline/segmenters/yolo_hybrid.py
209      index d9334f1..2e2e1fe 100644
210      --- a/backend/pipeline/segmenters/yolo_hybrid.py
211      +++ b/backend/pipeline/segmenters/yolo_hybrid.py
212      @@ -22,11 +22,14 @@ import concurrent.futures
213      import logging
214      import os
215      import time
216      -from typing import Any, Dict, List, Tuple
217      +from typing import Any, Dict, List, Tuple, Union
218
219      from backend.pipeline.detectors.person_detector import PersonDetector
220      from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
221      from backend.providers import provider_registry
222      +
223      +# Standardized Success|Failure contract: an unrunnable config is NOT a 0-rally result.
224      +from backend.results import Failure
225      from backend.sports import sport_registry
226      from backend.utils.paths import output_base
227      from backend.utils.video import extract_video_chunk, get_video_duration
228      @@ -63,9 +66,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
229      def _resolve_provider(self, video_path: str):
230          """Phase 0: resolve sport profile + AI provider (with API key).
231
232      - Returns the configured provider instance, or ``None`` to signal that
233      - ``process_video`` should early-return ``[]`` (bad sport, missing API key,
234      - or unknown provider). Behaviour is identical to the inlined version.
235      + Returns the configured provider instance, or a ``Failure`` (bad sport, missing
API key,
236      + or unknown provider) that ``process_video`` propagates ? so an unrunnable
config is marked
237      + FAILED with a reason instead of a silent 0-rally "success" (the wf294 loud-
failures bar;
238      + mirrors the gemini/trajectory segmenters). A genuine "ran, found nothing" is
unaffected.
239          """
240          # Get sport profile config
241          sport_name = self.config.sport
242      @@ -73,7 +77,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
243          sport_registry.get(sport_name)
244          except KeyError as e:
245              logger.error(str(e))
246          - return None
247          + return Failure(str(e))
248

```

```

249         # Get AI provider and model config
250         provider_name = self.config.ai_provider
251     @@ -85,12 +89,13 @@ class HybridYoloSegmenter(BasePlaySegmenter):
252         if not api_key:
253             from backend.pipeline.segmenters._keyhint import api_key_hint
254
255         -         logger.error(
256         +         msg = (
257             f"{api_key_env_var} environment variable is not set! "
258             f"To run the {provider_name} segmenter, set your API key. "
259             + api_key_hint(api_key_env_var)
260         )
261         -         return None
262         +         logger.error(msg)
263         +         return Failure(msg, is_recoverable=True)
264
265         try:
266             provider = provider_registry.get(
267     @@ -98,7 +103,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
268         )
269         except KeyError as e:
270             logger.error(str(e))
271         -         return None
272         +         return Failure(str(e))
273
274         logger.info(
275             f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}"
276     @@ -320,9 +325,13 @@ class HybridYoloSegmenter(BasePlaySegmenter):
277         chunk_duration: float,
278         handoff_padding: float,
279         ai_max_workers: int,
280         -     ) -> List[dict]:
281         +     ) -> "Union[List[dict], Failure]":
282             """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and
ask
283         -         the provider for semantic boundaries, in parallel. Verbatim move."""
284         +         the provider for semantic boundaries, in parallel.
285         +
286         +         Returns the confirmed segments, or a ``Failure`` when EVERY candidate window
failed to
287         +         yield a verdict (provider error / clip extraction failed) ? an incomplete run
has zero
288         +         information and must NOT be recorded as a 0-rally "success" (the wf294
class)."""
289             gemini_segments = []
290             gemini_dir = os.path.join(
291                 output_base(self.config), f"chunks_{provider_name}_handoff"
292     @@ -333,7 +342,11 @@ class HybridYoloSegmenter(BasePlaySegmenter):
293                 f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows..."
294             )
295
296         -         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
297         +         def process_candidate_window(
298         +             wi: int, window: Dict[str, Any]
299         +         ) -> Tuple[List[dict], bool]:
300         +             """Returns (valid_segments, errored) ? errored=True when this window
yielded NO verdict
301         +             (clip extraction failed, or the provider returned metrics['error'])."""
302             w_start = max(0, window["start"] - handoff_padding)
303             w_end = min(video_duration, window["end"] + handoff_padding)
304             w_dur = w_end - w_start
305     @@ -344,7 +357,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
306             video_path, w_start, w_dur, w_path, reencode=True

```

```

307         )
308         if not success:
309             -         return []
310             +         return [], True # extraction failed ? no verdict for this window
311
312         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
313         segments, metrics = provider.analyze_video(
314             @@ -353,7 +366,8 @@ class HybridYoloSegmenter(BasePlaySegmenter):
315                 # B2: a provider failure returns [] with metrics["error"] set ? that is
316                 # "we don't know", NOT "no rallies in this window". Surface it loudly so a
317                 # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
318             -         if metrics.get("error"):
319             +         errored = bool(metrics.get("error"))
320             +         if errored:
321                 logger.warning(
322                     "[Handoff %d] provider error ? window %d skipped: %s",
323                     wi + 1,
324             @@ -380,7 +394,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
325                 except OSError:
326                     pass
327
328             -         return valid_segments
329             +         return valid_segments, errored
330
331         with concurrent.futures.ThreadPoolExecutor(
332             max_workers=ai_max_workers
333             @@ -390,8 +404,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
334             for wi, w in enumerate(all_candidate_windows)
335             ]
336             # Iterate in submission order to preserve temporal ordering
337             -         for future in futures:
338             -             gemini_segments.extend(future.result())
339             +         window_results = [future.result() for future in futures]
340             +         for valid, _errored in window_results:
341             +             gemini_segments.extend(valid)
342             +         handoff_failed = sum(1 for _valid, errored in window_results if errored)
343
344             try:
345                 os.rmdir(gemini_dir)
346             @@ -401,6 +417,17 @@ class HybridYoloSegmenter(BasePlaySegmenter):
347                 logger.info(
348                     f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic play
segments."
349                 )
350             +         # wf294 (all-errored handoff): if EVERY candidate window failed to yield a
verdict
351             +         # (provider error / clip extraction failed), the run has zero information ? a
FAILURE, not
352             +         # a 0-rally match. A window that RAN and returned no rallies keeps this from
firing.
353             +         if all_candidate_windows and handoff_failed == len(all_candidate_windows):
354             +             msg = (
355             +                 f"yolo_hybrid AI handoff produced no verdict: all {handoff_failed}
candidate "
356             +                 "window(s) errored (provider error / clip extraction failed) ?
incomplete run, "
357             +                 "not a 0-rally match."
358             +             )
359             +             logger.error(msg)
360             +             return Failure(message=msg, is_recoverable=True)
361             return gemini_segments
362
363         def _populate_db(
364             @@ -456,19 +483,23 @@ class HybridYoloSegmenter(BasePlaySegmenter):

```



```

365
366         return segments_data
367
368     - def process_video(
369     + def process_video( # type: ignore[override]
370         self,
371         video_id: str,
372         video_path: str,
373         player_pool: List[str] = None,
374         max_frames: int = None,
375     - ) -> List[Dict[str, Any]]:
376     + ) -> "Union[List[Dict[str, Any]], Failure]":
377     +     # Result contract: an unrunnable config (bad sport/provider, missing key) or an
unreadable
378     +     # video returns a Failure so the run is marked failed ? NOT a silent 0-rally
"success"
379     +     # (the wf294 class). A genuine empty match still returns a bare []. Every
caller branches
380     +     # on isinstance(result, Failure).
381     self.person.lazy_load_model()
382
383     -     # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
384     +     # Phase 0: resolve sport profile + AI provider (returns a Failure on any config
failure).
385     provider = self._resolve_provider(video_path)
386     -     if provider is None:
387     -         return []
388     +     if isinstance(provider, Failure):
389     +         return provider
390
391     # Re-read the config values needed downstream (pure deterministic reads).
392     idx_cfg = self.config.indexing
393     @@ -479,7 +510,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
394     video_duration = get_video_duration(video_path)
395     if video_duration <= 0:
396         logger.error("Invalid video duration.")
397     -     return []
398     +     # An unreadable/broken video, not a 0-rally match.
399     +     return Failure(
400     +         message=f"yolo_hybrid: invalid/unreadable video duration
({video_duration})."
401     +     )
402
403     chunk_duration = idx_cfg.chunk_duration
404     chunk_overlap = idx_cfg.chunk_overlap
405     @@ -537,8 +571,8 @@ class HybridYoloSegmenter(BasePlaySegmenter):
406     except Exception:
407         pass
408
409     -     # Phase 3: AI Provider Handoff
410     -     gemini_segments = self._run_ai_handoff(
411     +     # Phase 3: AI Provider Handoff (returns a Failure when every window errored ?
wf294).
412     +     handoff = self._run_ai_handoff(
413         video_id,
414         video_path,
415         video_duration,
416     @@ -551,9 +585,11 @@ class HybridYoloSegmenter(BasePlaySegmenter):
417         handoff_padding,
418         ai_max_workers,
419     )
420     +     if isinstance(handoff, Failure):
421     +         return handoff
422

```

```

423         # Phase 4: DB Population
424     -     return self._populate_db(video_id, gemini_segments, model_name)
425     +     return self._populate_db(video_id, handoff, model_name)
426
427
428     # Register the segmenter
429     diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
430     index 461d7ca..88c50f6 100644
431     --- a/backend/worker/__main__.py
432     +++ b/backend/worker/__main__.py
433     @@ -114,6 +114,42 @@ def _write_report_json(
434         )
435
436
437     +def _serialize_and_push_outputs(
438     +    scratch: str,
439     +    out_uri: str,
440     +    video_id: str,
441     +    segments: List[dict],
442     +    status: str,
443     +    storage_cfg: dict,
444     +    video_uri: str = "",
445     +) -> List[str]:
446     +    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
447     +    push both to
448     +    ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
449     +    Shared by the SUCCESS path and the failure path (``_fail``) so a detector
450     +    timeout/abort leaves a
451     +    ``status="failed"`` report.json in the bucket ? the control plane's source of truth
452     +    (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
453     +    non-``success``
454     +    report as "no usable result" ? retry, and the restart-recovery poll waits for
455     +    report.json to
456     +    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. """
457     +    out_local = os.path.join(scratch, "outputs", video_id)
458     +    os.makedirs(out_local, exist_ok=True)
459     +    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
460     +    _write_report_json(
461     +        video_id,
462     +        segments,
463     +        status,
464     +        os.path.join(out_local, "report.json"),
465     +        video_uri=video_uri,
466     +    )
467     +    out_prefix = out_uri.rstrip("/")
468     +    pushed: List[str] = []
469     +    for fn in sorted(os.listdir(out_local)):
470     +        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
471     +        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
472     +        pushed.append(uri)
473     +    return pushed
474
475
476     def _run_startup_checks(config: Any) -> bool:
477     +    """M5 per-profile startup checks for the worker (the compute process, so it probes
478     +    CUDA
479     +    best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
480     +    aborts
481
482     @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
483         )
484
485
486     def _fail(rc: int) -> int:
487     -    # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
488     -    so a remote

```

```

481 -      # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
482 -      # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
483 +      # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
484 +      # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
485 +      # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
486 +      # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
487 +      # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
488 +      # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
489 +      # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
490 +      try:
491 +          _serialize_and_push_outputs(
492 +              scratch,
493 +              args.out_uri,
494 +              video_id,
495 +              [],
496 +              "failed",
497 +              storage_cfg,
498 +              str(getattr(args, "video_uri", "") or ""),
499 +          )
500 +      except Exception as e: # never mask the real failure on an artifact-push
hiccup
501 +          logger.warning("[worker] could not push failure artifacts: %s", e)
502 +      # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
503 +      if getattr(args, "done_uri", None):
504 +          _write_done_marker(
505 +              args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
506 @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
507     )
508
509     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
510 -    out_local = os.path.join(scratch, "outputs", video_id)
511 -    os.makedirs(out_local, exist_ok=True)
512 -    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
513 -    _write_report_json(
514 +    pushed = _serialize_and_push_outputs(
515 +        scratch,
516 +        args.out_uri,
517 +        video_id,
518 +        segments,
519 +        status,
520 -        os.path.join(out_local, "report.json"),
521 -        video_uri=str(getattr(args, "video_uri", "") or ""),
522 +        storage_cfg,
523 +        str(getattr(args, "video_uri", "") or ""),
524     )
525
526 -    out_prefix = args.out_uri.rstrip("/")
527 -    pushed = []
528 -    for fn in sorted(os.listdir(out_local)):
529 -        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
530 -        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
531 -        pushed.append(uri)
532 -
533     print(
534         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"

```

```

535     )
536     diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
537     index 64909e7..c157ed5 100644
538     --- a/tests/test_tracknet_hybrid.py
539     +++ b/tests/test_tracknet_hybrid.py
540     @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
541         from unittest.mock import MagicMock, patch
542
543         from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
544     +from backend.results import Failure
545
546
547     def _window(start=1.0, end=4.0):
548     @@ -91,7 +92,7 @@ def test_pipeline_happy_path(
549
550
551     @_patched
552     -def test_pipeline_no_api_key_returns_empty(
553     +def test_pipeline_no_api_key_is_failure(
554         mock_runner_cls,
555         mock_probe,
556         mock_dur,
557     @@ -99,12 +100,16 @@ def test_pipeline_no_api_key_returns_empty(
558         mock_provider_registry,
559         mock_env,
560     ):
561     +     """A missing API key (with skip_ai_handoff=False) is an unrunnable misconfig ? a
Failure, NOT a
562     +     0-rally []. This is a wf294-class silent-failure the worker must mark failed, not
"success"."""
563         mock_env.return_value = None
564         mock_dur.return_value = 30.0
565         mock_runner_cls.return_value = _make_runner()
566
567         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
568     -     assert seg.process_video("v1", "clip.mp4") == []
569     +     res = seg.process_video("v1", "clip.mp4")
570     +     assert isinstance(res, Failure) and res.is_recoverable
571     +     assert "API_KEY" in res.message
572
573
574     @_patched
575     @@ -124,7 +129,10 @@ def test_pipeline_healthcheck_failure_aborts(
576
577         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
578         res = seg.process_video("v1", "clip.mp4")
579     -     assert res == []
580     +     # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
581     +     # instead of writing an empty "success".
582     +     assert isinstance(res, Failure)
583     +     assert "environment not ready" in res.message
584     +     # Should bail before ever running inference.
585         mock_runner_cls.return_value.run_predict.assert_not_called()
586
587     @@ -144,7 +152,36 @@ def test_pipeline_no_trajectory_aborts(
588         mock_provider_registry.get.return_value = MagicMock()
589
590         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
591     -     assert seg.process_video("v1", "clip.mp4") == []
592     +     res = seg.process_video("v1", "clip.mp4")
593     +     # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
594     +     # this is exactly the wf294 silent-failure that the caller must now mark

```

```

failed/retryable.
595     +     assert isinstance(res, Failure)
596     +     assert "no trajectory" in res.message and res.is_recoverable
597     +
598     +
599     +@_patched
600     +def test_pipeline_zero_windows_is_legit_empty_not_failure(
601     +     mock_runner_cls,
602     +     mock_probe,
603     +     mock_dur,
604     +     mock_extract,
605     +     mock_provider_registry,
606     +     mock_env,
607     +):
608     +     """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
609     +     parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
610     +     (0-rally "success"), NOT a Failure."""
611     +     mock_env.return_value = "dummy_key"
612     +     mock_dur.return_value = 30.0
613     +     mock_extract.return_value = True
614     +     mock_runner_cls.return_value = _make_runner(windows=[]) # trajectory ok, zero
windows
615     +     mock_provider_registry.get.return_value = MagicMock()
616     +
617     +     seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
618     +     res = seg.process_video("v1", "clip.mp4")
619     +     assert res == [] and not isinstance(res, Failure)
620     +     # The detector DID run (a genuine empty, not an abort).
621     +     mock_runner_cls.return_value.run_predict.assert_called_once()
622
623
624     @_patched
625     diff --git a/tests/test_trajectory_hybrid_equivalence.py
b/tests/test_trajectory_hybrid_equivalence.py
626     index 0080f4d..0270e41 100644
627     --- a/tests/test_trajectory_hybrid_equivalence.py
628     +++ b/tests/test_trajectory_hybrid_equivalence.py
629     @@ -14,6 +14,7 @@ import pytest
630     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
631     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
632     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
633     +from backend.results import Failure
634
635     TWINS = [
636         (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
637     @@ -111,8 +112,10 @@ def test_provider_offsets_applied_identically(cls, family,
detector):
638
639     @pytest.mark.parametrize("cls,family,detector", TWINS)
640     def test_provider_failure_is_surfaced_not_silent(cls, family, detector, caplog):
641     -     """B2/P17: a provider failure ([], metrics['error']) must be logged loudly, not
silently
642     -     treated as 'no rallies in this window'. The run still returns [] (no analysis
done)."""
643     +     """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
loudly, not
644     +     silently treated as 'no rallies'. When it errors on the ONLY candidate window the
whole run has
645     +     zero verdict, so process_video now returns a Failure (was a bare [] that looked
like a 0-rally
646     +     match) ? the worker marks it failed/retryable instead of a silent empty
'success'."""
647     import logging

```

```

648
649     provider = MagicMock()
650     @@ -139,13 +142,45 @@ def test_provider_failure_is_surfaced_not_silent(cls, family,
detector, caplog):
651         logging.WARNING, logger="backend.pipeline.segmenters.trajectory_hybrid"
652     ):
653         res = seg.process_video("v1", "clip.mp4")
654     - assert res == [] # no analysis for the failed window
655     + assert isinstance(res, Failure) and res.is_recoverable # all (1) windows errored
-> no verdict
656     assert any(
657         "provider error" in r.message and "generate failed" in r.message
658         for r in caplog.records
659     ), [r.message for r in caplog.records]
660
661
662     +@pytest.mark.parametrize("cls,family,detector", TWINS)
663     +def test_partial_handoff_error_is_a_success_not_a_failure(cls, family, detector):
664     +     """The boundary of F3: when SOME candidate windows error but at least one RAN (even
if it found
665     +     no rallies), the run is NOT a failure ? only an all-errored handoff is. Here two
windows are
666     +     handed off; the first returns a rally, the second errors -> a Success with the
first rally."""
667     +     provider = MagicMock()
668     +     provider.analyze_video.side_effect = [
669     +         ([{"start_time": 0.5, "end_time": 2.0}], {}), # window 0: a real verdict
670     +         ([], {"error": "generate failed after retries: 503"}), # window 1: errored
671     +     ]
672     +     db = MagicMock()
673     +     db.add_candidate_segment.side_effect = [55, 56]
674     +     db.add_play_segment.return_value = 200
675     +     two_windows = [_window(1.0, 4.0), _window(10.0, 13.0)]
676     +     with patch.object(
677     +         cls, "_build_runner", return_value=(_runner(two_windows), {"weights_path":
"w"}))
678     +     ), patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), patch(
679     +         "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
680     +         return_value=30.0,
681     +     ), patch(
682     +         "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
683     +         return_value=True,
684     +     ), patch(
685     +         "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
686     +     ) as preg, patch("os.environ.get", return_value="dummy_key"):
687     +         preg.get.return_value = provider
688     +         seg = cls(db, {"indexing": {}})
689     +         res = seg.process_video("v1", "clip.mp4")
690     +         assert not isinstance(res, Failure)
691     +         assert len(res) == 1 and res[0]["start_time"] == 0.5
692     +
693     +
694     def test_base_is_not_registered():
695         from backend.pipeline.segmenters.base import segmenter_registry
696
697     diff --git a/tests/test_worker.py b/tests/test_worker.py
698     index 96edbee..8fb5e16 100644
699     --- a/tests/test_worker.py
700     +++ b/tests/test_worker.py
701     @@ -10,6 +10,7 @@ import json
702     import pytest
703
704     from backend.config import load_config
705     +from backend.results import Failure
706     from backend.worker import __main__ as wmain

```

```

707
708 # ----- pure helpers
709 @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
710
711
712 class _EmptySeg:
713 - """A segmenter that finds nothing ? the cold-start failure mode."""
714 + """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""
715
716     def __init__(self, db, config):
717         pass
718 @@ -200,6 +201,20 @@ class _EmptySeg:
719     return []
720
721
722 +class _FailSeg:
723 + """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
724 + trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
725 +
726 +     def __init__(self, db, config):
727 +         pass
728 +
729 +     def process_video(self, video_id, path, max_frames=None):
730 +         return Failure(
731 +             message="WASB detector produced no trajectory (timed out or aborted)",
732 +             is_recoverable=True,
733 +         )
734 +
735 +
736     def test_setup_logging_levels():
737         import logging
738
739 @@ -246,12 +261,112 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
740     assert "0 segments" in msgs and "vidZ" in msgs
741     assert "detector_impl=native" in msgs # surfaces the resolved detector
742     assert "native runner UNAVAILABLE" in msgs # points at the likely cause
743 -     assert (
744 -         json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
745 -             "segment_count"
746 +     # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
747 +     # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
748 +     report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
749 +     assert report["segment_count"] == 0 and report["status"] == "success"
750 +
751 +
752 +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
753 +    tmp_path, monkeypatch
754 +):
755 +     """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
756 +     as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
757 +     status="failed" so the control plane / alpha-user surface can show an error /
retry."""
758 +     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
759 +     monkeypatch.setattr(
760 +         wmain.validator_registry,
761 +         "run_all_with_context",
762 +         lambda path, cfg: ([_OKResult()], {}),
763 +     )
764 +     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)

```

```

765 + monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
766 + video = tmp_path / "in.mp4"
767 + video.write_bytes(b"FAKE")
768 + out = tmp_path / "bucket"
769 + done = tmp_path / "bucket" / "_dispatch" / "tok.done"
770 +
771 + rc = wmain.main(
772 +     [
773 +         "infer",
774 +         "--video-uri",
775 +         str(video),
776 +         "--out-uri",
777 +         str(out),
778 +         "--scratch",
779 +         str(tmp_path / "s"),
780 +         "--segmenter",
781 +         "wasb_hybrid",
782 +         "--done-uri",
783 +         str(done),
784 +     ]
785 + )
786 + # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
787 + assert rc == 3
788 + # report.json is written as an explicit FAILURE, not a false empty "success".
789 + report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
790 + assert report["status"] == "failed" and report["segment_count"] == 0
791 + assert report["segments"] == []
792 + # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
793 + marker = json.loads(done.read_text())
794 + assert marker["returncode"] == 3 and marker["status"] == "failed"
795 + assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
796 +
797 +
798 +def test_cmd_infer_real_trajectory_segmenter_failure_flows_to_report(tmp_path,
monkeypatch):
799 +     """End-to-end: the ACTUAL WasbHybridSegmenter no-trajectory abort (the wf294 shape)
flows
800 +     through cmd_infer -> _fail(3) -> a status="failed" report.json + done-marker. Locks
the two
801 +     halves (segmenter Failure + worker report/marker) TOGETHER, not just via the
_FailSeg stub."""
802 +     from unittest.mock import MagicMock
803 +
804 +     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
805 +
806 +     runner = MagicMock()
807 +     runner.healthcheck.return_value = (True, "env ok")
808 +     runner.run_predict.return_value = None # WASB timed out / aborted: no trajectory
produced
809 +
810 +     # Force the real base process_video down to its no-trajectory abort without a
GPU/video.
811 +     monkeypatch.setattr(WasbHybridSegmenter, "_resolve_runner", lambda self, cfg:
(runner, {}))
812 +     monkeypatch.setattr(
813 +         WasbHybridSegmenter, "_probe_fps_width", lambda *a, **k: (30.0, 1280.0)
814 +     )
815 +     monkeypatch.setattr(
816 +         "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration", lambda p:
30.0
817 +     )
818 +     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidR")
819 +     monkeypatch.setattr(
820 +         wmain.validator_registry, "run_all_with_context", lambda path, cfg:

```



```

([_OKResult()], {})
821     + )
822     + monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name:
WasbHybridSegmenter)
823     + monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
824     +
825     + video = tmp_path / "in.mp4"
826     + video.write_bytes(b"FAKE")
827     + out = tmp_path / "bucket"
828     + done = tmp_path / "bucket" / "_dispatch" / "tok.done"
829     +
830     + rc = wmain.main(
831     +     [
832     +         "infer",
833     +         "--video-uri",
834     +         str(video),
835     +         "--out-uri",
836     +         str(out),
837     +         "--scratch",
838     +         str(tmp_path / "s"),
839     +         "--segmenter",
840     +         "wasb_hybrid",
841     +         "--done-uri",
842     +         str(done),
843     +         "--set",
844     +         "indexing.skip_ai_handoff=true", # no API key needed; isolate the detector
abort
845     +     ]
846     -     == 0
847     )
848     + assert rc == 3
849     + report = json.loads((out / "outputs" / "vidR" / "report.json").read_text())
850     + assert report["status"] == "failed" and report["segment_count"] == 0
851     + marker = json.loads(done.read_text())
852     + assert marker["returncode"] == 3 and marker["status"] == "failed"
853
854
855     def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
856         @@ -266,20 +381,30 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path,
monkeypatch):
857             video.write_bytes(b"X")
858             cfg = tmp_path / "config.json"
859             cfg.write_text("{}")
860             + out = tmp_path / "o"
861             + done = tmp_path / "o" / "_dispatch" / "tok.done"
862             rc = wmain.main(
863             +     [
864             +         "infer",
865             +         "--video-uri",
866             +         str(video),
867             +         "--out-uri",
868             -         str(tmp_path / "o"),
869             +         str(out),
870             +         "--config",
871             +         str(cfg),
872             +         "--scratch",
873             +         str(tmp_path / "s"),
874             +         "--done-uri",
875             +         str(done),
876             +     ]
877             )
878             assert rc == 2
879             + # A validation failure is ALSO written as an explicit failure (not a missing
artifact / false
880             + # success): report.json status="failed" + the done-marker rc 2 / status="failed".

```

```

881 + report = json.loads((out / "outputs" / "vidF" / "report.json").read_text())
882 + assert report["status"] == "failed" and report["segment_count"] == 0
883 + marker = json.loads(done.read_text())
884 + assert marker["returncode"] == 2 and marker["status"] == "failed"
885
886
887 # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push
bundle ---
888 diff --git a/tests/test_yolo_hybrid.py b/tests/test_yolo_hybrid.py
889 index 38e78b2..09da6c2 100644
890 --- a/tests/test_yolo_hybrid.py
891 +++ b/tests/test_yolo_hybrid.py
892 @@ -11,6 +11,7 @@ import os
893     from unittest.mock import MagicMock, patch
894
895     from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
896 +from backend.results import Failure
897
898
899     def _make_cap_mock(num_frames=300):
900 @@ -110,8 +111,10 @@ def test_yolo_hybrid_segmenter(
901     def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
902         mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap, caplog
903     ):
904 -        """B2/P17: a provider failure ([], metrics['error']) must be logged loudly, not
905         silently
906         - treated as 'no rallies'. The run still returns [] (no analysis done)."""
907 +        """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
908         loudly, not
909         + silently treated as 'no rallies'. When it errors on the ONLY candidate window the
910         whole run has
911         + zero verdict, so process_video now returns a Failure (was a bare [] that looked
912         like a 0-rally
913         + match) ? the worker marks it failed/retryable instead of a silent empty
914         'success'."""
915     import logging
916
917     mock_env_get.return_value = "dummy_api_key"
918 @@ -147,13 +150,49 @@ def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
919     logging.WARNING, logger="backend.pipeline.segmenters.yolo_hybrid"
920 ):
921     res = segmenter.process_video("v1", "dummy.mp4")
922 - assert res == [] # no analysis for the failed window
923 + assert isinstance(res, Failure) and res.is_recoverable # all (1) windows errored
924 -> no verdict
925     assert any(
926         "provider error" in r.message and "generate failed" in r.message
927         for r in caplog.records
928     ), [r.message for r in caplog.records]
929
930
931 +@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
932 +@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
933 +@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
934 +@patch("os.environ.get")
935 +def test_yolo_hybrid_partial_handoff_error_is_a_success_not_a_failure(
936     mock_env_get, mock_provider_registry, mock_extract, mock_get_dur
937 ):
938     """The boundary of F3: SOME windows errored but at least one RAN, so the run is a
939     Success (the
940     + first window's rally), NOT a Failure. Stage 2 is patched to hand off two windows
941     directly."""
942     mock_env_get.return_value = "dummy_api_key"
943     mock_get_dur.return_value = 30.0
944     mock_extract.return_value = True

```

```

937 +
938 + mock_provider = MagicMock()
939 + mock_provider.analyze_video.side_effect = [
940 +     ([{"start_time": 0.5, "end_time": 2.0}], {}), # window 0: a real verdict
941 +     ([], {"error": "generate failed after retries: 503"}), # window 1: errored
942 + ]
943 + mock_provider_registry.get.return_value = mock_provider
944 +
945 + db_mock = MagicMock()
946 + db_mock.add_play_segment.return_value = "yolo_segment_1"
947 + two_windows = [{"start": 1.0, "end": 4.0}, {"start": 10.0, "end": 13.0}]
948 +
949 + segmenter = HybridYoloSegmenter(db_mock, {"indexing": {"use_gpu": False}})
950 + segmenter.person.model = MagicMock() # short-circuit lazy model load
951 + with patch.object(
952 +     HybridYoloSegmenter, "_collect_candidate_windows", return_value=two_windows
953 + ), patch.object(
954 +     HybridYoloSegmenter, "_persist_candidates", return_value=[None, None]
955 + ):
956 +     res = segmenter.process_video("v1", "dummy.mp4")
957 +     assert not isinstance(res, Failure)
958 +     assert len(res) == 1 and res[0]["start_time"] == 0.5
959 +
960 +
961 + @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
962 + @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
963 + @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
964 + @@ -162,7 +201,8 @@ def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
965 +     def test_yolo_hybrid_no_api_key(
966 +         mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
967 +     ):
968 +         - """Segmenter should return empty when GEMINI_API_KEY is missing."""
969 +         + """A missing API key is an unrunnable misconfig ? a Failure, NOT a 0-rally [] that
the worker
970 +         + would write as a silent status="success" (the wf294 class; yolo_hybrid is the
default segmenter)."""
971 +         mock_env_get.return_value = None
972 +
973 +         db_mock = MagicMock()
974 +         @@ -172,7 +212,8 @@ def test_yolo_hybrid_no_api_key(
975 +             segmenter.person.model = MagicMock() # short-circuit lazy model load
976 +
977 +             res = segmenter.process_video("v1", "dummy.mp4")
978 +             - assert res == []
979 +             + assert isinstance(res, Failure) and res.is_recoverable
980 +             + assert "API_KEY" in res.message
981 +
982 +
983 + @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
984 +

```

4. assistant (2026-07-06T18:00:51.533Z)

I have the full diff. Now let me read the actual source files to verify the F3 logic in context ? particularly what happens between `handoff\_indices` selection and the guard, and how `\_select\_for\_handoff` interacts with `all\_candidate\_windows`.

5. user (2026-07-06T18:00:53.521Z)

```

350             f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
351             f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
352             f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
353         )
354

```

```

355         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
356         # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
357         # persistence and the handoff gate below.
358         window_stats = [
359             self._enrich_candidate_stats(
360                 cw, points, fps, frame_width, video_path, idx_cfg
361             )
362             for cw in all_candidate_windows
363         ]
364
365         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
366         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
367         if store_candidates:
368             for i, cw in enumerate(all_candidate_windows):
369                 candidate_ids[i] = self.db.add_candidate_segment(
370                     video_id,
371                     detector=self.name,
372                     start_time=cw["start"],
373                     end_time=cw["end"],
374                     chunk_index=cw["chunk_index"],
375                     confidence=min(1.0, cw["peak_velocity"]),
376                     stats=window_stats[i],
377                     detection_params=detection_params,
378                 )
379
380         if not all_candidate_windows:
381             return []
382
383         # --- Phase 3: AI provider handoff over padded candidate clips ---
384         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
385         # candidates above stay the full superset ? only the served play_segments are
gated.
386         handoff_indices = self._select_for_handoff(
387             all_candidate_windows, window_stats, idx_cfg
388         )
389
390         ai_segments: List[dict] = []
391         if skip_ai:
392             # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
393             # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
394             # falls back to the duration-based estimate (no AI exchange count).
395             ai_segments = [
396                 {
397                     "start_time": all_candidate_windows[wi]["start"],
398                     "end_time": all_candidate_windows[wi]["end"],
399                     "shuttle_exchange_count": None,
400                     "shot_count_confidence": "LocalNoAI",
401                     "_candidate_id": candidate_ids[wi],
402                 }
403                 for wi in handoff_indices
404             ]
405             logger.info(
406                 "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
"
407                 "(no AI handoff).",
408                 len(ai_segments),
409             )
410         else:
411             handoff_dir = os.path.join(
412                 output_base(self.config), f"chunks_{provider_name}_handoff"

```

```

413         )
414     os.makedirs(handoff_dir, exist_ok=True)
415     logger.info(
416         "Phase 3: %s validation on %d candidate windows...",
417         provider_name,
418         len(handoff_indices),
419     )
420
421     def process_candidate_window(
422         wi: int, window: Dict[str, Any]
423     ) -> Tuple[List[dict], bool]:
424         """Returns (valid_segments, errored) ? errored=True when this window
yielded NO
425         verdict (clip extraction failed, or the provider returned
metrics['error'])."""
426         w_start = max(0, window["start"] - handoff_padding)
427         w_end = min(video_duration, window["end"] + handoff_padding)
428         w_dur = w_end - w_start
429         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
430         if not extract_video_chunk(
431             video_path, w_start, w_dur, w_path, reencode=True
432         ):
433             return [], True # extraction failed ? no verdict for this window
434         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
435         segments, metrics = provider.analyze_video(
436             w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
437         )
438         # B2: a provider failure returns [] with metrics["error"] set ? that is
439         # "we don't know", NOT "no rallies in this window". Surface it loudly so
a
440         # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
441         errored = bool(metrics.get("error"))
442         if errored:
443             logger.warning(
444                 "[Handoff %d] provider error ? window %d skipped: %s",
445                 wi + 1,
446                 wi,
447                 metrics["error"],
448             )
449         valid = []
450         for seg in segments:
451             try:
452                 seg["start_time"] = float(seg["start_time"]) + w_start
453                 seg["end_time"] = float(seg["end_time"]) + w_start
454                 seg["_candidate_id"] = candidate_ids[wi]
455                 valid.append(seg)
456             except (ValueError, TypeError, KeyError) as e:
457                 logger.warning(
458                     "Dropping segment with unparseable timestamps: %s ? %s",
459                     seg,
460                     e,
461                 )
462         try:
463             os.remove(w_path)
464         except OSError:
465             pass
466         return valid, errored
467
468     with concurrent.futures.ThreadPoolExecutor(
469         max_workers=ai_max_workers
470     ) as executor:
471         futures = [
472             executor.submit(
473                 process_candidate_window, wi, all_candidate_windows[wi]

```

```

474         )
475         for wi in handoff_indices
476     ]
477     window_results = [future.result() for future in futures]
478     for valid, _errored in window_results:
479         ai_segments.extend(valid)
480     handoff_failed = sum(1 for _valid, errored in window_results if errored)
481     try:
482         os.rmdir(handoff_dir)
483     except OSError:
484         pass
485     logger.info(
486         "Phase 3 completed. AI confirmed %d semantic play segments.",
487         len(ai_segments),
488     )
489     # wf294 (all-errored handoff): if EVERY handed-off window failed to yield a
verdict
490     # (provider error / clip extraction failed), the run has zero information ?
a FAILURE,
491     # not a 0-rally match. A single window that RAN and returned no rallies is a
legit empty
492     # and keeps this from firing. (skip_ai has no provider, so it never reaches
here.)
493     if handoff_indices and handoff_failed == len(handoff_indices):
494         msg = (
495             f"{label} AI handoff produced no verdict: all {handoff_failed}
candidate "
496             "window(s) errored (provider error / clip extraction failed) ?
incomplete "
497             "run, not a 0-rally match."
498         )
499         logger.error(msg)
500         return Failure(message=msg, is_recoverable=True)
501
502     # --- Phase 4: DB population + candidate linking ---
503     segments_data = []
504     for segment in ai_segments:
505         try:
506             start = float(segment["start_time"])
507             end = float(segment["end_time"])
508             dur = end - start
509             metadata = {
510                 "shot_count": self._shot_count_from(segment, dur),
511                 "shot_count_confidence": segment.get(
512                     "shot_count_confidence", "Unknown"
513                 ),
514                 "detector": f"{self.family}-hybrid-{model_name}",
515             }
516             segment_id = self.db.add_play_segment(
517                 video_id, start, end, None, None, metadata
518             )
519             if segment_id:
520                 candidate_id = segment.get("_candidate_id")
521                 if candidate_id is not None:
522                     self.db.link_candidate_to_segment(candidate_id, segment_id)
523                 segments_data.append(
524                     {
525                         "id": segment_id,
526                         "start_time": start,
527                         "end_time": end,
528                         "duration": dur,
529                         "metadata": metadata,

```

```

300         # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
301         confidence = min(1.0, w["peak_velocity"])
302         candidate_ids[i] = self.db.add_candidate_segment(
303             video_id,
304             detector="yolo_hybrid",
305             start_time=w["start"],
306             end_time=w["end"],
307             chunk_index=w["chunk_index"],
308             confidence=confidence,
309             stats=stats,
310             detection_params=detection_params,
311         )
312
313     return candidate_ids
314
315     def _run_ai_handoff(
316         self,
317         video_id: str,
318         video_path: str,
319         video_duration: float,
320         all_candidate_windows: List[Dict[str, Any]],
321         candidate_ids: List[Any],
322         provider,
323         sport_profile,
324         provider_name: str,
325         chunk_duration: float,
326         handoff_padding: float,
327         ai_max_workers: int,
328     ) -> "Union[List[dict], Failure]":
329         """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and ask
330         the provider for semantic boundaries, in parallel.
331
332         Returns the confirmed segments, or a ``Failure`` when EVERY candidate window
333         failed to
334         yield a verdict (provider error / clip extraction failed) ? an incomplete run
335         has zero
336         information and must NOT be recorded as a 0-rally "success" (the wf294
337         class)."""
338         gemini_segments = []
339         gemini_dir = os.path.join(
340             output_base(self.config), f"chunks_{provider_name}_handoff"
341         )
342         os.makedirs(gemini_dir, exist_ok=True)
343
344         logger.info(
345             f"Starting Phase 3: AI Provider ({provider_name}) Validation on
346             {len(all_candidate_windows)} candidate windows..."
347         )
348
349         def process_candidate_window(
350             wi: int, window: Dict[str, Any]
351         ) -> Tuple[List[dict], bool]:
352             """Returns (valid_segments, errored) ? errored=True when this window yielded
353             NO verdict
354
355             (clip extraction failed, or the provider returned metrics['error'])."""
356             w_start = max(0, window["start"] - handoff_padding)
357             w_end = min(video_duration, window["end"] + handoff_padding)
358             w_dur = w_end - w_start
359
360             w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
361             # Re-encode short handoff clips to avoid keyframe-boundary drift
362             success = extract_video_chunk(
363                 video_path, w_start, w_dur, w_path, reencode=True
364             )

```

```

359         if not success:
360             return [], True # extraction failed ? no verdict for this window
361
362         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
363         segments, metrics = provider.analyze_video(
364             w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
365         )
366         # B2: a provider failure returns [] with metrics["error"] set ? that is
367         # "we don't know", NOT "no rallies in this window". Surface it loudly so a
368         # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
369         errored = bool(metrics.get("error"))
370         if errored:
371             logger.warning(
372                 "[Handoff %d] provider error ? window %d skipped: %s",
373                 wi + 1,
374                 wi,
375                 metrics["error"],
376             )
377
378         # Coordinate mapping ? shift local timestamps back to full-video time
379         valid_segments = []
380         for segment in segments:
381             try:
382                 segment["start_time"] = float(segment["start_time"]) + w_start
383                 segment["end_time"] = float(segment["end_time"]) + w_start
384                 # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
385                 segment["_candidate_id"] = candidate_ids[wi]
386                 valid_segments.append(segment)
387             except (ValueError, TypeError, KeyError) as e:
388                 logger.warning(
389                     f"Dropping segment with unparseable timestamps: {segment} ? {e}"
390                 )
391
392         try:
393             os.remove(w_path)
394         except OSError:
395             pass
396
397         return valid_segments, errored
398
399     with concurrent.futures.ThreadPoolExecutor(
400         max_workers=ai_max_workers
401     ) as executor:
402         futures = [
403             executor.submit(process_candidate_window, wi, w)
404             for wi, w in enumerate(all_candidate_windows)
405         ]
406         # Iterate in submission order to preserve temporal ordering
407         window_results = [future.result() for future in futures]
408         for valid, _errored in window_results:
409             gemini_segments.extend(valid)
410         handoff_failed = sum(1 for _valid, errored in window_results if errored)
411
412         try:
413             os.rmdir(gemini_dir)
414         except OSError:
415             pass
416
417         logger.info(
418             f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic play
segments."
419         )
420         # wf294 (all-errored handoff): if EVERY candidate window failed to yield a

```



```

verdict
421         # (provider error / clip extraction failed), the run has zero information ? a
FAILURE, not
422         # a 0-rally match. A window that RAN and returned no rallies keeps this from
firing.
423         if all_candidate_windows and handoff_failed == len(all_candidate_windows):
424             msg = (
425                 f"yolo_hybrid AI handoff produced no verdict: all {handoff_failed}
candidate "
426                 "window(s) errored (provider error / clip extraction failed) ?
incomplete run, "
427                 "not a 0-rally match."
428             )
429             logger.error(msg)

```

7. assistant (2026-07-06T18:00:58.051Z)

Now I need to understand the critical interaction: in trajectory_hybrid, the guard uses `\_handoff\_indices` (the `\_select\_for\_handoff` subset). Let me look at `\_select\_for\_handoff` to understand whether it can return an empty subset while `all\_candidate\_windows` is non-empty ? the fusion Gate scenario the lens explicitly asks about.

8. user (2026-07-06T18:00:59.649Z)

```

backend\infrastructure\database_user_models.py:21:    # (`fusion_hybrid._select_for_handoff`) is
the NEXT, owner-gated PR. NULL user_id =
backend\infrastructure\database.py:335:            # resolves (the FUTURE owner-gated wiring into
`_select_for_handoff`). This slice
backend\pipeline\personalization\__init__.py:7:wires it into
``fusion_hybrid._select_for_handoff``. See ``serving.py``.
backend\pipeline\personalization\serving.py:21:? NOT WIRED INTO THE SERVED PATH.
``fusion_hybrid._select_for_handoff`` still reads only the
backend\pipeline\segmenters\fusion_hybrid.py:16:- ``_select_for_handoff`` ? optional served Gate
A/B: when ``indexing.fusion.min_crossings``
backend\pipeline\segmenters\fusion_hybrid.py:199:    def _select_for_handoff(
backend\pipeline\segmenters\trajectory_hybrid.py:161:    def _select_for_handoff(
backend\pipeline\segmenters\trajectory_hybrid.py:386:        handoff_indices =
self._select_for_handoff(
backend\eval\rally_seg_eval.py:225:    NEVER-EMPTY safeguard (mirrors
``FusionSegmenter._select_for_handoff``): if the gate would
backend\eval\served_gate_a.py:11:``FusionSegmenter._select_for_handoff``. THIS module drives
exactly those production methods ?
backend\eval\served_gate_a.py:70:    ``net_crossings``) and ``_select_for_handoff`` (Gate A) ?
over windows from the production
backend\eval\served_gate_a.py:95:    keep = seg._select_for_handoff(windows, window_stats,
idx_cfg)

```

9. assistant (2026-07-06T18:01:06.461Z)

Two implementations of `\_select\_for\_handoff`: the base one in trajectory_hybrid.py (line 161) and an override in fusion_hybrid.py (line 199). Let me read both, plus the eval reference to the "NEVER-EMPTY safeguard" which is the key clue.

10. user (2026-07-06T18:01:07.972Z)

```

161     def _select_for_handoff(
162         self,
163         windows: List[Dict[str, Any]],
164         window_stats: List[Dict[str, Any]],
165         idx_cfg: "IndexingConfig",
166     ) -> List[int]:
167         """Indices of the candidate windows that proceed to the AI handoff (and thus may
168         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
169         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.

```

```

170         Persisted candidates are NOT affected ? they remain the full superset for
offline
171         re-scoring."""
172         return list(range(len(windows)))
173
174     def process_video( # type: ignore[override]
175         self,
176         video_id: str,
177         video_path: str,
178         player_pool: Optional[List[str]] = None,
179         max_frames: Optional[int] = None,
180     ) -> "Union[List[Dict[str, Any]], Failure]":

```

11. user (2026-07-06T18:01:08.720Z)

```

199     def _select_for_handoff(
200         self,
201         windows: List[Dict[str, Any]],
202         window_stats: List[Dict[str, Any]],
203         idx_cfg: IndexingConfig,
204     ) -> List[int]:
205         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
206         (0) keep every window ? identical to the substrate. Persisted candidates are
207         untouched (full superset).
208
209         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
210         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
211         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
212         set rather than emit a video with zero rallies. This makes Gate A/B strictly
213         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
214         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
215         below it always wins on an otherwise-empty video. Observed on real footage
216         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
217         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
218         fcfg = idx_cfg.fusion
219         min_crossings = int(fcfg.min_crossings)
220         min_players = int(fcfg.min_players)
221         person_on = bool(fcfg.cue_flags.get("person", False))
222
223         keep: List[int] = []
224         for i, st in enumerate(window_stats):
225             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
226                 continue
227             if (
228                 person_on
229                 and min_players > 0
230                 and st.get("players_in_court", 0.0) < min_players
231             ):
232                 continue
233             keep.append(i)
234
235         if not keep and window_stats:
236             # Gating emptied a non-empty video ? fall back to the ungated set.
237             return list(range(len(window_stats)))
238         return keep
239
240
241     segmenter_registry.register(FusionSegmenter)

```

12. assistant (2026-07-06T18:01:21.148Z)

The fusion Gate has a NEVER-EMPTY safeguard: if `all\_candidate\_windows` is non-empty, `\_select\_for\_handoff` always returns a non-empty subset. So the `handoff\_indices` empty-while-`all\_candidate\_windows`-non-empty scenario cannot occur in the fusion path, and the base returns all. Good ? that means `handoff\_indices` is empty ****iff**** `all\_candidate\_windows` is empty. And the code already guards `if not all\_candidate\_windows: return []` at line 380 before ever reaching the handoff. So `handoff\_indices` is non-empty whenever we reach the guard. The guard `handoff\_indices and handoff\_failed == len(handoff\_indices)` cannot fire a false 0==0 Failure. That path is correct.

Now let me scrutinize the ordering/thread-safety concern and the `\_candidate\_id` handling. Let me check whether the yolo_hybrid `handoff` (which may be a `Failure`) is passed to `\_populate\_db`, and confirm the caller contract. Also let me examine the `skip\_ai` path in yolo (does it have one?) and confirm F3 doesn't touch partial-success paths. Let me also verify the `add\_candidate\_segment.side\_effect = [55, 56]` test ? is `add\_candidate\_segment` even called in the trajectory partial test given `store\_candidates`?

Let me check `store\_candidates` default and the yolo `process\_video` flow around the handoff return, plus confirm there's no yolo skip_ai path.

13. user (2026-07-06T18:01:22.506Z)

```

483
484         return segments_data
485
486     def process_video( # type: ignore[override]
487         self,
488         video_id: str,
489         video_path: str,
490         player_pool: List[str] = None,
491         max_frames: int = None,
492     ) -> "Union[List[Dict[str, Any]], Failure]":
493         # Result contract: an unrunnable config (bad sport/provider, missing key) or an
unreadable
494         # video returns a Failure so the run is marked failed ? NOT a silent 0-rally
"success"
495         # (the wf294 class). A genuine empty match still returns a bare []. Every caller
branches
496         # on isinstance(result, Failure).
497         self.person.lazy_load_model()
498
499         # Phase 0: resolve sport profile + AI provider (returns a Failure on any config
failure).
500         provider = self._resolve_provider(video_path)
501         if isinstance(provider, Failure):
502             return provider
503
504         # Re-read the config values needed downstream (pure deterministic reads).
505         idx_cfg = self.config.indexing
506         sport_profile = sport_registry.get(self.config.sport)
507         provider_name = self.config.ai_provider
508         model_name = self.config.ai_model
509
510         video_duration = get_video_duration(video_path)
511         if video_duration <= 0:
512             logger.error("Invalid video duration.")
513             # An unreadable/broken video, not a 0-rally match.
514             return Failure(
515                 message=f"yolo_hybrid: invalid/unreadable video duration
({video_duration})."
516             )
517

```

```

518     chunk_duration = idx_cfg.chunk_duration
519     chunk_overlap = idx_cfg.chunk_overlap
520     velocity_thresh = (
521         idx_cfg.yolo_velocity_threshold
522     ) # Fraction of frame width per second
523     merge_gap = idx_cfg.dead_time_merge_gap
524     frame_skip = idx_cfg.yolo_frame_skip
525     tracker_config = idx_cfg.yolo_tracker
526     handoff_padding = idx_cfg.ai_handoff_padding
527     ai_max_workers = idx_cfg.ai_max_workers
528     log_candidates = idx_cfg.log_yolo_candidates
529     store_candidates = idx_cfg.store_yolo_candidates
530
531     # Snapshot of the params that produced these detections ? stored alongside each
532     # candidate so a fine-tuning run can reproduce / correlate against them.
533     detection_params = {
534         "velocity_thresh": velocity_thresh,
535         "merge_gap": merge_gap,
536         "frame_skip": frame_skip,
537         "tracker": tracker_config,
538         "detector_model": idx_cfg.detector_model,
539         "detector_score_threshold": idx_cfg.detector_score_threshold,
540         "min_action_window_duration": idx_cfg.min_action_window_duration,
541     }
542
543     chunks_dir = os.path.join(output_base(self.config), "chunks_yolo")
544     os.makedirs(chunks_dir, exist_ok=True)
545
546     # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.
547     all_candidate_windows = self._collect_candidate_windows(
548         video_id,
549         video_path,
550         video_duration,
551         idx_cfg,
552         chunk_duration,
553         chunk_overlap,
554         velocity_thresh,
555         merge_gap,
556         frame_skip,
557         chunks_dir,
558     )
559
560     # Phase 2b: log + persist raw candidates (window index -> candidate_id).
561     candidate_ids = self._persist_candidates(
562         video_id,
563         all_candidate_windows,
564         detection_params,
565         log_candidates,
566         store_candidates,
567     )
568
569     try:
570         os.rmdir(chunks_dir)
571     except Exception:
572         pass
573
574     # Phase 3: AI Provider Handoff (returns a Failure when every window errored ?
575     handoff = self._run_ai_handoff(
576         video_id,
577         video_path,
578         video_duration,
579         all_candidate_windows,
580         candidate_ids,
581         provider,

```

```

582         sport_profile,
583         provider_name,
584         chunk_duration,
585         handoff_padding,
586         ai_max_workers,
587     )
588     if isinstance(handoff, Failure):
589         return handoff
590
591     # Phase 4: DB Population
592     return self._populate_db(video_id, handoff, model_name)
593
594
595     # Register the segmenter
596     segmenter_registry.register(HybridYoloSegmenter)
597

```

14. assistant (2026-07-06T18:01:30.868Z)

Now there's a critical thing to check in yolo: when `all\_candidate\_windows` is empty, what does `\_run\_ai\_handoff` return? Let me trace: with empty `all\_candidate\_windows`, `futures` is empty, `window\_results` is empty, `handoff\_failed = 0`, and the guard `if all\_candidate\_windows and ...` is `False` (empty list is falsy), so it returns `gemini\_segments = []`. Then `\_populate\_db(video\_id, [], model\_name)` returns `[]`. Correct ? legit empty stays a bare `[]`. Good.

But wait ? in yolo, `\_run\_ai\_handoff` does NOT have a `skip\_ai` guard. In the trajectory version, the `skip\_ai` branch bypasses the handoff. But does yolo_hybrid support `skip\_ai`? Let me check whether yolo reads `skip\_ai\_handoff` at all.

15. user (2026-07-06T18:01:32.591Z)

No matches found

16. assistant (2026-07-06T18:01:39.871Z)

yolo_hybrid has no skip_ai path ? always uses the provider, so F3's assumption ("skip_ai never reaches this path") holds for yolo since there is no skip_ai at all. Good.

Now let me verify the caller contract at the three cited callsites to confirm `Failure` is handled and there's no regression where a `Failure` could be treated as a truthy list. Let me check `worker.cmd\_infer:326`, `main.py:1557`, `orchestration.py:806`.

17. user (2026-07-06T18:01:40.685Z)

```

300         return rc
301
302     segments: List[dict] = []
303     segmenter_actual = None
304     segmentation_ran = False
305     status = "failed"
306     try:
307         if failures:
308             print(
309                 "[worker] validation FAILED: "
310                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
311             )
312             return _fail(2)
313     seg_name = args.segmenter or config.indexing.default_segmenter
314     segmenter_cls = segmenter_registry.get(seg_name)
315     if not segmenter_cls:
316         print(
317             f"[worker] unknown segmenter: {seg_name!r} "
318             f"(have {list(segmenter_registry.list_available())})"

```

```

319         )
320         return _fail(2)
321     segmenter_actual = seg_name
322     result = segmenter_cls(db, config).process_video(
323         video_id, local_video, max_frames=args.max_frames
324     )
325     segmentation_ran = True
326     if isinstance(result, Failure):
327         print(f"[worker] segmenter FAILED: {result.message}")
328         return _fail(3)
329     segments = result.value if hasattr(result, "value") else result
330     status = "success"
331     # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
332     # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
333     # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
334     if not segments:
335         detector_impl = (
336             os.environ.get("RALLY_DETECTOR_IMPL")
337             or getattr(config.indexing, "detector_impl", None)
338             or "auto"
339         )
340         logger.warning(
341             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
342             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
343             "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
344             "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
345             "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
346             "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
347             "detections, or the input resolution differs from the tuned proxy
resolution. "
348             "Re-run with -v for the native command + frame counts.",
349             video_id,
350             segmenter_actual,
351             detector_impl,
352         )
353     finally:
354         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
355         emit_indexer_report(
356             db=db,
357             video_id=video_id,
358             video_path=local_video,
359             config=config,
360             val_results=val_results,
361             val_context=val_context,
362             segmenter_requested=args.segmenter,
363             segmenter_actual=segmenter_actual,
364             was_reingest=False,
365             segmentation_ran=segmentation_ran,
366         )
367
368     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
369     pushed = _serialize_and_push_outputs(

```

18. assistant (2026-07-06T18:01:46.021Z)

The worker caller correctly branches on `isinstance(result, Failure)` at line 326 and returns

`\_fail(3)`. Note line 329 `result.value if hasattr(result, 'value')` ? since `Failure` is already handled above, `result` is a bare list here, which doesn't have `.value`, so `segments = result`. Fine.

Now let me verify the trajectory equivalence test's `add\_candidate\_segment.side\_effect = [55, 56]`. In the trajectory partial test, does `store\_candidates` default to True so `add\_candidate\_segment` is called for both windows? If `store\_candidates` is False, `candidate\_ids` stays `[None, None]` and `add\_candidate\_segment` is never called ? the `side\_effect` would just be unused (not an error). Let me check the default and how `\_runner`/`\_window` are defined in that test to make sure the two-window partial test actually exercises two handoff windows.

19. user (2026-07-06T18:01:47.827Z)

```
1      """Equivalence contract for the trajectory-hybrid twins.
2
3      tracknet_hybrid and wasb_hybrid were copy-paste twins that drifted (the
4      shot_count parse); both are now thin subclasses of TrajectoryHybridSegmenter.
5      These tests run BOTH through the same scenarios and assert identical P3/P4
6      behavior, so any future divergence has to be introduced deliberately, in the
7      base, for both at once.
8      """
9
10     from unittest.mock import MagicMock, patch
11
12     import pytest
13
14     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
15     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
16     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
17     from backend.results import Failure
18
19     TWINS = [
20         (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
21         (WasbHybridSegmenter, "wasb", "wasb_hybrid"),
22     ]
23
24
25     def _window(start=1.0, end=4.0):
26         return {
27             "start": start,
28             "end": end,
29             "active_frame_count": 30,
30             "peak_velocity": 0.4,
31             "mean_velocity": 0.2,
32             "track_id_count": 1,
33             "chunk_index": 0,
34         }
35
36
37     def _runner(windows=None):
38         runner = MagicMock()
39         runner.healthcheck.return_value = (True, "env ok")
40         runner.run_predict.return_value = "out.csv"
41         runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
42         runner.trajectory_to_action_windows.return_value = (
43             windows if windows is not None else [_window()]
44         )
45         return runner
46
47
48     def _run(cls, provider_segments, db=None):
49         """Run one segmenter class through the standard mocked pipeline."""
50         provider = MagicMock()
51         provider.analyze_video.return_value = (provider_segments, {})
```

```

52     db = db or MagicMock()
53     db.add_candidate_segment.return_value = 55
54     db.add_play_segment.return_value = 200
55
56     with patch.object(
57         cls, "_build_runner", return_value=(_runner(), {"weights_path": "w"})
58     ), patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), patch(
59         "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
60         return_value=30.0,
61     ), patch(
62         "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
63         return_value=True,
64     ), patch(
65         "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
66     ) as preg, patch("os.environ.get", return_value="dummy_key"):
67         preg.get.return_value = provider
68         seg = cls(db, {"indexing": {}})
69         return db, seg.process_video("v1", "clip.mp4")
70
71
72 @pytest.mark.parametrize("cls,family,detector", TWINS)
73 def test_detector_identity_flows_into_db_rows(cls, family, detector):
74     db, res = _run(
75         cls, [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": 4}]
76     )
77     assert len(res) == 1
78     _, kwargs = db.add_candidate_segment.call_args
79     assert kwargs["detector"] == detector
80     assert kwargs["detection_params"]["detector"] == detector
81     # P4 metadata tags the family + model.
82     assert res[0]["metadata"]["detector"].startswith(f"{family}-hybrid-")
83     db.link_candidate_to_segment.assert_called_once_with(55, 200)
84
85
86 @pytest.mark.parametrize("cls,family,detector", TWINS)
87 @pytest.mark.parametrize(
88     "exchange,expected",
89     [
90         (4, 4), # provider-reported count wins
91         ("7", 7), # numeric strings coerce
92         (None, 3), # explicit null -> duration estimate (2.5s/0.8 -> 3 floor)
93         ("garbage", 3), # unparseable -> same estimate
94     ],
95 )
96 def test_shot_count_behavior_is_identical(cls, family, detector, exchange, expected):
97     """The historical drift point: wasb_hybrid relied on int(None) raising into
98     the fallback while tracknet_hybrid short-circuited. One implementation now."""
99     seg_dict = {"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": exchange}
100     _, res = _run(cls, [seg_dict])
101     assert res[0]["metadata"]["shot_count"] == expected
102
103
104 @pytest.mark.parametrize("cls,family,detector", TWINS)
105 def test_provider_offsets_applied_identically(cls, family, detector):
106     # Window [1,4] + 2.0s default padding -> clip starts at max(0, -1) = 0.0;
107     # provider-relative 0.5 stays 0.5 absolute here.
108     _, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
109     assert res[0]["start_time"] == 0.5
110     assert res[0]["end_time"] == 3.0
111
112
113 @pytest.mark.parametrize("cls,family,detector", TWINS)
114 def test_provider_failure_is_surfaced_not_silent(cls, family, detector, caplog):
115     """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
loudly, not

```


20. assistant (2026-07-06T18:02:00.961Z)

The tests are well-formed. Now let me examine the critical ordering/thread-safety concern more carefully, and one subtle edge: the `\_candidate\_id` linkage under the new ordering. In the trajectory partial test, `add\_candidate\_segment.side\_effect = [55, 56]` and `db.add\_candidate\_segment` ? is `store\_candidates` on by default so both are consumed? If the default is off, `candidate\_ids = [None, None]` and neither is consumed, but that's harmless. Let me verify the trajectory happy-path test uses only one segment and consumes one, and confirm the `\_populate\_db` ordering.

More importantly, let me focus on the real potential defect: **ordering under the new** `[f.result() for f in futures]` collection. In the OLD code, results were extended in submission order via `for future in futures: extend(future.result())`. In the NEW code, `window\_results = [future.result() for future in futures]` also preserves submission order (list comprehension over `futures` in order), then iterates `for valid, \_errored in window\_results`. So ordering is identical. No regression there.

Let me now verify one thing the lens flagged: is there any shared-state race? `ai\_segments`/`gemini\_segments` are extended AFTER the executor block (outside threads), and `candidate\_ids[wi]` is only read inside threads (each thread reads a distinct `wi`). No shared mutable writes inside threads. Safe.

Now let me double check the yolo test's assertion depends on `\_populate\_db` returning the segment with `start\_time`. Let me verify `\_populate\_db` preserves `start\_time` in its return dicts.

21. user (2026-07-06T18:02:02.403Z)

```
430         return Failure(message=msg, is_recoverable=True)
431     return gemini_segments
432
433     def _populate_db(
434         self, video_id: str, gemini_segments: List[dict], model_name: str
435     ) -> List[Dict[str, Any]]:
436         """Phase 4: DB Population ? write confirmed AI segments, link them back to their
437         source candidate, and build the returned segments_data. Verbatim move."""
438         segments_data = []
439         for segment in gemini_segments:
440             try:
441                 start = float(segment["start_time"])
442                 end = float(segment["end_time"])
443                 dur = end - start
444
445                 shot_count = segment.get("shuttle_exchange_count")
446                 if shot_count is None:
447                     shot_count = max(3, int(dur / 0.8))
448                 else:
449                     try:
450                         shot_count = int(shot_count)
451                     except (ValueError, TypeError):
452                         shot_count = max(3, int(dur / 0.8))
453
454                 metadata = {
455                     "shot_count": shot_count,
456                     "shot_count_confidence": segment.get(
457                         "shot_count_confidence", "Unknown"
458                     ),
459                     "detector": f"yolo-hybrid-{model_name}",
460                 }
461
462                 segment_id = self.db.add_play_segment(
463                     video_id, start, end, None, None, metadata
464                 )
465                 if segment_id:
466                     # Link back to the candidate that produced this segment. A candidate
```

```

467             # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
468             # the last one ? the link table records "this candidate was
confirmed".
469             candidate_id = segment.get("_candidate_id")
470             if candidate_id is not None:
471                 self.db.link_candidate_to_segment(candidate_id, segment_id)
472             segments_data.append(
473                 {
474                     "id": segment_id,
475                     "start_time": start,
476                     "end_time": end,
477                     "duration": dur,
478                     "metadata": metadata,
479                 }
480             )
481         except Exception as e:
482             logger.warning(f"Skipping invalid segment struct: {e}")
483
484         return segments_data

```

22. assistant (2026-07-06T18:02:16.397Z)

This confirms the yolo partial test correctly asserts ``res[0]["start_time"] == 0.5``.

Now let me carefully consider the ONE genuinely subtle edge case the lens raises about the `**denominator` and ``errored`` semantics** for a case not covered: `**what if `extract_video_chunk` returns a clip but the provider returns segments AND `metrics['error']` is set?**` Then ``errored=True`` but ``valid`` may be non-empty (segments got appended). That's fine ? the window contributed segments AND is counted as errored. But this can only affect the all-errored guard if EVERY window is errored. If every window is errored but some returned segments, then ``ai_segments`` is non-empty yet the run returns ``Failure``, discarding those segments. Is that correct? Per the contract, ``metrics['error']`` means "we don't know" ? a provider error. If the provider returned partial segments alongside an error, treating the whole run as a `Failure` (retryable) is the conservative correct choice consistent with the stated B2 semantics. Not a defect.

Now, the most important scenario to verify: `**legit-empty must NOT fail.**` Consider trajectory with ``all_candidate_windows`` non-empty, handoff runs, all windows return ``[]`` with NO error (a real "no rallies" verdict). Then ``handoff_failed = 0``, guard ``handoff_failed == len(handoff_indices)`` is ``0 == N`` ? `False`. Returns ``[]``. Correct.

Let me now verify there isn't a defect in how ``_populate_db`` is reached in trajectory when ``ai_segments`` is empty but NOT all-errored ? it falls through to Phase 4 with empty ``ai_segments`` ? returns ``[]``. Correct.

Let me check the base ``TrajectoryHybridSegmenter.process_video`` return type is ``Union[List, Failure]`` and the ``# type: ignore[override]`` ? the review says trajectory_hybrid is NOT mypy-quarantined and must stay clean. Let me verify there's no ``# type: ignore`` on the base ``process_video`` that would be flagged, and check whether ``_populate_db`` in trajectory returns a bare list (it does, line 502+). The base class ABC return type ? let me confirm the override annotation is consistent.

Let me verify one final potential real issue: in trajectory, line 174 ``def process_video( # type: ignore[override]`` ? but the diff context (line 28-32) shows the override annotation was changed to ``Union[List, Failure]`` WITHOUT a ``# type: ignore[override]`` in the diff. Let me check the actual current line 174.

23. user (2026-07-06T18:02:17.843Z)

```

174     def process_video( # type: ignore[override]
175         self,
176         video_id: str,
177         video_path: str,
178         player_pool: Optional[List[str]] = None,

```

```

179         max_frames: Optional[int] = None,
180     ) -> "Union[List[Dict[str, Any]], Failure]":
181         # Result contract (the ABC declares Success|Failure). Anything that means the
run could
182         # NOT actually complete ? an unrunnable config (unknown sport/provider, missing
API key),
183         # an unreadable video, an unavailable detector (native-not-supported), the env
healthcheck
184         # not ready, or run_predict timing out / aborting with no trajectory ? returns a
``Failure``
185         # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match
(a trajectory
186         # was produced but no candidate/AI-confirmed rallies) still returns a bare
``[]``. Every
187         # caller (worker.cmd_infer, main.py, orchestration) branches on
isinstance(result, Failure).

```

24. assistant (2026-07-06T18:02:23.213Z)

The `# type: ignore[override]` is present on the base process_video (line 174). This is fine and pre-existing.

Now let me do a final check on the ABC signature to ensure the `# type: ignore[override]` is warranted (i.e., mypy would otherwise complain about the return type widening / signature). Let me look at the base `BasePlaySegmenter.process\_video` signature. This matters because trajectory_hybrid must stay mypy-clean, and if the ABC actually declares `Union[List, Failure]` (Success|Failure), then a `# type: ignore[override]` on a compatible override could be an **unused ignore** ? mypy error under `warn\_unused\_ignores`. Let me check.

25. user (2026-07-06T18:02:24.161Z)

```

1     from abc import ABC, abstractmethod
2     from typing import Any, Dict, List, Optional, Type
3
4     from backend.infrastructure.database import Database
5     from backend.results import (
6         Result, # new standardized contract (Result[Value] == Success[Value] | Failure)
7     )
8
9
10    def _normalize_config(config: Any) -> Any:
11        """Normalize a plain-dict config into an AppConfig so segmenters can use typed
12        attribute access consistently. Production always passes AppConfig; tests may
13        pass plain dicts ? this bridge keeps both paths working."""
14        if isinstance(config, dict):
15            from backend.config.models import AppConfig
16
17            try:
18                return AppConfig(**config)
19            except Exception:
20                return (
21                    config # invalid config ? let the segmenter's own validation reject it
22                )
23        return config
24
25
26    class BasePlaySegmenter(ABC):
27        def __init__(self, db: Database, config: Dict[str, Any]):
28            self.db = db
29            self.config = _normalize_config(config)
30
31        @property
32        @abstractmethod
33        def name(self) -> str:
34            """Unique identifier for this segmenter."""

```

```

35         pass
36
37     @property
38     @abstractmethod
39     def description(self) -> str:
40         """Human-readable explanation of this segmenter."""
41         pass
42
43     @abstractmethod
44     def process_video(
45         self,
46         video_id: str,
47         video_path: str,
48         player_pool: List[str] = None,
49         max_frames: int = None,
50     ) -> "Result[List[Dict[str, Any]]]":
51         """
52         Processes a video to detect play segments, populates SQLite database,
53         and returns a Result.
54
55         Use isinstance(result, Failure) (or result.is_success()) to distinguish
56         "no segments found" from "something went wrong" (missing key, healthcheck
57         failure, etc.). This is the start of the standardized error/result contract.
58         """
59         pass
60
61
62     class SegmenterRegistry:
63         def __init__(self):
64             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
65
66         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
67             """Registers a new segmenter class under its name property."""
68             try:
69                 temp_inst = segmenter_cls(None, {})
70                 name = temp_inst.name
71             except Exception as e:
72                 raise ValueError(
73                     f"Failed to resolve name property from segmenter class
74 {segmenter_cls.__name__}: {e}"
75                 ) from e
76             self._segmenters[name] = segmenter_cls
77
78         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
79             return self._segmenters.get(name)
80
81         def list_available(self) -> Dict[str, str]:
82             """Returns a dict of registered segmenter names and their descriptions."""
83             res = {}
84             for name, cls in self._segmenters.items():
85                 try:
86                     temp_inst = cls(None, {})
87                     desc = temp_inst.description
88                 except Exception:
89                     desc = "No description available."
90             res[name] = desc
91             return res
92
93     segmenter_registry = SegmenterRegistry()
94

```

26. assistant (2026-07-06T18:02:34.886Z)

The ABC declares `Result[List[Dict[str, Any]]]` which is `Success[List] | Failure`. The override

returns `Union[List[Dict[str, Any]], Failure]` ? a bare list, NOT `Success[List]`. So the override return type is genuinely incompatible with the ABC (a bare `List` is not a `Success[List]`), which is exactly why `# type: ignore[override]` is warranted and NOT unused. This is consistent and mypy-clean. No defect.

The `# type: ignore[override]` on yolo's `process\_video` (line 486) is fine since yolo is mypy-quarantined anyway.

Let me do one last verification of the concern the lens raised about whether the trajectory denominator is right in a scenario I haven't fully closed: **fusion Gate reduces `handoff\_indices` to a strict, non-empty subset of `all\_candidate\_windows`, then all of THOSE selected windows error.** Guard: `handoff\_indices` non-empty (say 2 selected out of 5 candidates), `handoff_failed == len(handoff_indices) ? 2 == 2 ? Failure. Is that correct? The selected handoff windows are the only ones that would become play_segments; if all of them errored, we truly have zero verdict on the servable set ? Failure is correct. The 3 gated-out candidates were never going to produce segments anyway. Correct denominator.

I've thoroughly traced both paths. Let me verify there are no other callers of these segmenters that might mishandle the new `Failure` ? specifically the `main.py:1557` and `orchestration.py:806` callsites ? to rule out a broken-contract defect.

27. user (2026-07-06T18:02:36.394Z)

```
1548         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1549         segmenter_actual = seg_name
1550         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1551         segmenter = segmenter_cls(db_instance, global_config)
1552         result = segmenter.process_video(
1553             video_id, local_video, max_frames=args.max_frames
1554         )
1555         segmentation_ran = True
1556
1557         if isinstance(result, Failure):
1558             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1559             print("[FAIL] Indexing halted for video.")
1560             segments = []
1561         else:
1562             segments = result.value if hasattr(result, "value") else result
1563             print(
1564                 f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}"
1565             )
1566             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1567             _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1568         finally:
1569             # Always emit the per-video observability report. Never raises.
1570             paths = emit_indexer_report(
1571                 db=db_instance,
1572                 video_id=video_id,
1573                 # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI
would write a
1574                 # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source
URI is kept
1575                 # in DB provenance via db.add_video.
1576                 video_path=local_video,
1577                 config=_report_config_for_input(global_config, args.video_path),
```

28. user (2026-07-06T18:02:37.083Z)

```
798         except Exception:
799             # A raising segmenter must not leave half-written rows: restore the pre-run
800             # state (same destroy-after-confirm contract as the Failure branch), then
let
801             # the job worker record the failure.
```

```

802         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
803         raise
804     segmentation_ran = True
805
806     if isinstance(result, Failure):
807         logger.error(f"[API] Segmenter failed: {result.message}")
808         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
809         response = {
810             "video_id": video_id,
811             "status": "failed",
812             "message": f"Segmenter '{seg_name}' failed: {result.message}",
813             "results": [r.model_dump() for r in val_results],
814             "play_segments": [],
815         }
816         return response
817
818     segments = result.value if hasattr(result, "value") else result
819     notify("finalizing")
820     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
821
822     response = {

```

29. assistant (2026-07-06T18:02:58.176Z)

All three callers correctly branch on ``isinstance(result, Failure)`` and handle it. The contract is intact end-to-end.

I have thoroughly reviewed the F3 all-errored-handoff logic in both files, traced ``_select_for_handoff`` (base + fusion NEVER-EMPTY safeguard), the denominator choices, ``errored`` semantics, ordering/thread-safety, the `skip_ai` and partial-success paths, and all three caller callsites.

Summary of my verification (no defects found):

1. ****Denominator correctness (trajectory uses ``handoff_indices``, yolo uses ``all_candidate_windows``)**** ? Both are correct. In trajectory, the ``if not all_candidate_windows: return []`` guard at line 380 executes before the handoff, and ``_select_for_handoff`` is either the base (returns ALL windows) or fusion's override which has a NEVER-EMPTY safeguard (fusion_hybrid.py:235-237: non-empty input ? non-empty output). So ``handoff_indices`` is empty iff ``all_candidate_windows`` is empty, and we never reach the guard with an empty ``handoff_indices``. The ``0 == 0`` false-Failure cannot fire. In yolo, ``_run_ai_handoff`` uses ``all_candidate_windows`` directly with an ``if all_candidate_windows and ...`` guard, so empty candidates return a bare ``[]`` (falsy guard). Correct.

2. ****The fusion-Gate-filters-all scenario**** ? Cannot mislabel: the NEVER-EMPTY safeguard guarantees ``handoff_indices`` non-empty when ``all_candidate_windows`` non-empty, and the guard correctly fails only when every SELECTED (servable) window errored, which is the right servable-set denominator.

3. ****`errored` semantics**** ? A window that ran the provider and returned ``[]`` with no ``metrics['error']`` yields ``errored=False`` (a real "no rallies" verdict), correctly NOT counted toward ``handoff_failed``. ``errored=True`` only on extraction-fail or ``metrics['error']``. Correct per the B2 contract.

4. ****Ordering/thread-safety**** ? ``window_results = [f.result() for f in futures]`` preserves submission (temporal) order identically to the old ``for future in futures`` loop; ``ai_segments`/`gemini_segments`` are extended after the executor block (single-threaded), and threads only read distinct ``candidate_ids[wi]``. No shared-state race, no ordering regression.

5. ****skip_ai / partial-success unchanged**** ? trajectory's ``skip_ai`` branch bypasses the handoff entirely and never reaches the guard; yolo has no `skip_ai` path at all. Partial (``handoff_failed < len(windows)``) returns Success with the windows that ran (verified by the two new boundary tests). All-success (``handoff_failed == 0``) returns the bare list.

6. ****Contract intact**** ? All three callers (worker/`\_\_main\_\_`.py:326, main.py:1557,

orchestration.py:806) branch on `isinstance(result, Failure)`. The `# type: ignore[override]` on trajectory's `process\_video` is warranted (returns bare `List | Failure`, not `Success[List]`, so genuinely incompatible with the ABC's `Result[...]`) and therefore not an unused-ignore ? mypy stays clean.

I found no correctness, edge-case, contract, thread-safety, or test defects in the F3 logic.

30. user (2026-07-06T18:02:58.436Z)

Structured output provided successfully